PROJECT REPORT

D-RTOS

Defense Radar Real-Time Operating System Simulator

SUBMITTED BY :
TEAM MEMBERS-
AADITYA CHOURASIA-24BIT0556
MANAV MISHRA-24BIT0100

Abstract

D-RTOS is a simulated real-time Defense Radar Operating System that converts radar events into operating-system processes and dynamically manages their execution. A detected object (for example an unknown aircraft, missile or drone) is classified into a threat level, wrapped in a Process Control Block, assigned a priority and a deadline, and then passed through the complete OS pipeline: scheduling on a four-core processor, memory allocation and paging, resource allocation with deadlock avoidance, and synchronized access to a shared radar data buffer.

The project implements eight CPU scheduling policies, real-time EDF scheduling with deadline monitoring, multi-core load balancing, contiguous memory allocation (First/Best/Worst Fit), paging with FIFO and LRU page replacement, a Resource Allocation Graph with DFS cycle detection, the Banker's algorithm, and mutex/semaphore based synchronization with a controlled race-condition demonstration. Results are persisted through a MongoDB service layer that automatically falls back to an in-memory database, so the system never crashes when the database is offline. A tactical Tkinter command-center GUI with Matplotlib analytics visualizes every subsystem.

Keywords: Operating System, Real-Time Scheduling, EDF, Banker's Algorithm, Paging, Deadlock, Synchronization, Radar Simulation, Python, Tkinter.

1. Introduction

1.1.Background

An operating system is the layer of software that manages hardware resources and provides services to running programs. Its core responsibilities - process management, CPU scheduling, memory management, resource handling, deadlock control and synchronization - are normally studied as separate theoretical topics. Real-time systems add a further constraint: tasks must finish before a deadline, and a late result may be as bad as a wrong one.

A defense radar is a natural example of such a system. Radar sweeps continuously generate events that must be classified and processed immediately, with high-threat objects receiving the CPU first. D-RTOS uses this scenario as a unifying story so that every OS concept appears in a meaningful, connected workflow rather than as an isolated algorithm.

1.2.Problem Statement

Operating-system algorithms are difficult to understand from textbooks alone because their behavior unfolds over time. The problem addressed by this project is to design a single integrated simulator in which these algorithms can be executed, observed, compared and stored, using a realistic real-time use case.

1.3.Objectives

- Model radar detections as OS processes with complete Process Control Blocks (PCB) and a defined state life-cycle.
- Implement and compare eight CPU scheduling policies using standard performance metrics.
- Provide real-time (EDF) scheduling with deadline monitoring and miss detection.
- Simulate four CPU cores with an OS-level load balancer.
- Demonstrate contiguous allocation, paging and FIFO/LRU page replacement.
- Implement deadlock detection (RAG + DFS) and deadlock avoidance (Banker's algorithm).
- Demonstrate process synchronization with mutex and counting semaphore, including a race condition.
- Persist sessions in MongoDB with a crash-free in-memory fallback.
- Present all subsystems in an interactive command-center GUI.

1.4.Scope and Limitations of Scope

The system is an **educational simulation**. The radar, targets and threats are fictional, and time is simulated rather than driven by real hardware interrupts. Inter-process communication (IPC) is intentionally out of scope and the synchronization module focuses on mutual exclusion and bounded-buffer coordination only.

2. System Architecture

2.1.End-to-End Processing Pipeline

Every radar event travels through the following stages. Each stage maps to a module in the `core/` package, and the outcome of the run is saved by the database layer.

#	Stage	What happens	Module
1	Radar Event	Object (type, distance, speed, heading) is generated or injected manually	radar/simulator.py
2	Threat Classification	Event is mapped to a threat level using type, distance, speed and probability	radar/classifier.py
3	Process Creation	A process is created from the classified event	core/process.py
4	PCB Creation	PID, arrival/burst/remaining time, state, pages and resources are recorded	core/pcb.py
5	Priority + Deadline	Higher threat gives higher priority and a tighter deadline	core/process_manager.py
6	Resource Requirement	Process declares the resource instances it needs	core/resources.py
7	CPU Scheduling	Ready queue is ordered by the selected policy	core/scheduler.py, realtime.py
8	Multi-Core Execution	Tasks are dispatched to CPU-0..CPU-3; balancer migrates tasks	core/multicore.py
9	Memory Management	Contiguous partition and logical-to-physical page mapping	core/memory.py, paging.py
10	Synchronization	Shared radar buffer accessed via mutex and semaphore	core/synchronization.py
11	Deadlock / Banker Check	Request is granted only if the system stays in a safe state	core/deadlock.py, banker.py
12	Process Completion	Process becomes TERMINATED; resources and memory are released	core/process_manager.py
13	Performance Analysis	Waiting, turnaround, response, utilization, throughput computed	core/performance.py
14	MongoDB Storage	Session and metrics stored (or kept in memory if DB is offline)	database/mongodb.py

2.2.Layered Design

The project follows a layered architecture that separates presentation, simulation logic and persistence. Higher layers depend only on the layers beneath them, which keeps the OS engine testable without any GUI.

2.3.Technology Stack

Component	Technology	Purpose
Language	Python 3 (100% of the code base)	Implementation of all modules
GUI	Tkinter	Command-center screens, forms, canvases
Charts	Matplotlib	Scheduling comparison and analytics charts
Database	MongoDB via a service layer	Session history and metrics persistence
Fallback	In-memory store	Guarantees crash-free execution when MongoDB is offline

3. Project Structure

The source tree is organised by responsibility. The listing below is the complete directory layout of the repository.

```
D-RTOS/
|
|-- main.py                       # Entry point
|-- requirements.txt              # Dependency files
|-- core/                         # OS Simulation Engine Layer
|   |-- process.py                # Process enums and profile configs
|   |-- pcb.py                    # PCB Class definition
|   |-- process_manager.py        # Transitions and queues manager
|   |-- scheduler.py              # FCFS, SJF, Priority, RR, PP, SRTF, MLQ
|   |-- realtime.py               # EDF and Deadline monitor metrics
|   |-- multicore.py              # CPU 0-3 management and balancing
|   |-- memory.py                 # Contiguous Fit allocation block maps
|   |-- paging.py                 # Pages translation and FIFO/LRU tests
|   |-- resources.py              # Resource allocation request managers
|   |-- deadlock.py               # RAG graph cycle DFS checks
|   |-- banker.py                 # Banker's safety state matrix calculations
|   |-- synchronization.py        # Mutex, Semaphores and Radar Buffers
|   `-- performance.py            # Metric snapshots and aggregations
|
|-- radar/                        # Fictional Simulation Layer
|   |-- simulator.py              # Object sweep generators
|   `-- classifier.py             # Threat level mapping
|
|-- database/                     # Database Persistence
|   `-- mongodb.py                # Service layer with collections management
|
|-- gui/                          # Presentation views (Tkinter & Matplotlib)
|   |-- theme.py                  # Command center tactical styles
|   |-- main_window.py            # Navigation layout container
|   |-- dashboard.py              # System metrics overview
|   |-- radar_view.py             # PPI sweep and blips canvas
|   |-- process_view.py           # Process lists and PCB inspector
|   |-- scheduler_view.py         # Gantt canvas and metrics comparer
|   |-- realtime_view.py          # Deadline gauge tables
|   |-- multicore_view.py         # Core load progress HUD
|   |-- memory_view.py            # Physical RAM address partition graphs
|   |-- paging_view.py            # Logical translator and replacement stepper
|   |-- resource_view.py          # Allocators list and slider forms
|   |-- deadlock_view.py          # Deadlock cycle paths and Banker's safe sequence
|   |-- synchronization_view.py # Mutex locks and Race Condition demo
|   |-- analytics_view.py         # Matplotlib bar charts comparing schedules
|   |-- history_view.py           # History sessions browser
|   |-- scenario_view.py          # 8-Phase walkthrough runner
|   `-- settings_view.py          # DB Connection URI configuration
|
`-- tests/                        # Verification Tests Suite
```

Figure 2: Directory structure of the D-RTOS repository

3.1.Package Responsibilities

Package	Responsibility	Depends on
core/	Pure OS logic: processes, schedulers, memory, deadlock, synchronization, metrics. No GUI code.	Python standard library
radar/	Generates fictional radar objects and classifies them into threat levels; feeds the OS engine.	core/
database/	Isolates all MongoDB access behind MongoDBService; provides in-memory fallback.	pymongo (optional)
gui/	Tkinter screens, tactical theme and Matplotlib charts; calls into core/radar/ database.	core/, radar/, database/
tests/	Verification suite checking algorithm correctness.	core/, radar/, database/

4. Module-wise Design and Implementation

4.1.Process and PCB Manager (core/process.py, pcb.py, process_manager.py)

Each radar object becomes a process described by a **Process Control Block**. The PCB stores everything the OS needs to manage the process:

- **Identification:** process ID (PID) and process type (e.g. Unknown Aircraft).
- **Timing:** arrival time, burst time and remaining time.
- **Scheduling data:** priority, threat level and deadline.
- **Memory data:** logical page mapping.
- **Resources:** allocated and required resource footprint.

Process states follow the life-cycle `NEW -> READY -> RUNNING -> WAITING -> TERMINATED`. The Process Manager owns the queues and performs every legal transition, e.g. admitting a NEW process to READY, dispatching READY to RUNNING, blocking RUNNING to WAITING on a resource request, and terminating on completion.

4.2.CPU Scheduling (core/scheduler.py)

Eight scheduling policies are provided. Seven are implemented in `scheduler.py` and EDF is implemented in `realtime.py`. The user selects a policy and a time quantum (used by Round Robin) on the CPU Scheduling screen and presses *Run Scheduler* to obtain a Gantt chart and a metrics summary.

#	Policy	Type	Selection criterion	Remarks
1	FCFS	Non-preemptive	Earliest arrival	Simple; suffers from the convoy effect
2	SJF	Non-preemptive	Smallest burst time	Minimizes average waiting time; may starve long jobs
3	Priority	Non-preemptive	Highest priority	Threat level drives priority; risk of starvation
4	Round Robin	Preemptive	Time slice (quantum)	Fair; good response time; more context switches
5	Preemptive Priority	Preemptive	Highest priority, preempts lower	Critical threats interrupt routine tracking
6	SRTF	Preemptive	Shortest remaining time	Optimal average waiting time; high switching overhead
7	MLQ	Multi-queue	Critical: preemptive priority; Interactive: RR; Routine: FCFS	Matches radar task classes
8	EDF	Preemptive (real-time)	Earliest absolute deadline	Optimal for deadline-driven uniprocessor tasks

Round Robin algorithm (outline):

```
queue = ready processes sorted by arrival
time = 0
while queue not empty:
    p = queue.pop_front()
    run = min(quantum, p.remaining)
    record Gantt slice (p.pid, time, time + run)
    time += run;  p.remaining -= run
    enqueue newly arrived processes
    if p.remaining > 0: queue.push_back(p)       # preempted
    else: p.completion = time                    # finished
```

4.3.Real-Time Scheduling and Deadline Monitor (core/realtime.py)

In a hard real-time system a missed deadline is a failure. **Earliest Deadline First (EDF)** always dispatches the ready task whose absolute deadline is nearest. The deadline monitor compares each task's completion time with its deadline and reports met/missed counts and slack (deadline minus completion). The Real-Time Monitor screen shows these values as gauge tables.

4.4.Multi-Core Execution and Load Balancing (core/multicore.py)

The simulator models **four CPU cores (CPU-0 to CPU-3)** running concurrently. Each core has its own ready queue. The **OS load balancer** periodically compares queue lengths and migrates ready tasks from overloaded cores to underloaded cores so that utilization is even. The Multi-Core Util screen shows a per-core load HUD.

4.5.Memory Management (core/memory.py)

Physical memory is divided into partitions (a block map). When a process arrives, a partition is chosen by one of three placement strategies:

- **First Fit** - first hole large enough; fastest search.
- **Best Fit** - smallest hole that fits; leaves the smallest leftover.
- **Worst Fit** - largest hole; leaves the largest (more usable) leftover.

The Memory Alloc screen draws the physical RAM as an address-partition graph so that fragmentation is visible.

4.6.Virtual Memory and Paging (core/paging.py)

The **Paging Manager** maps logical addresses to physical frames using 32 MB page blocks. A logical address is split into a page number and an offset; the page table gives the frame number, and the physical address is *frame x page size + offset*. When memory is full a page-replacement policy chooses the victim page. The simulator runs **FIFO** and **LRU** on a page-reference string and compares their page-fault counts and fault rates; the Virtual Memory screen steps through the replacement visually.

4.7.Resource Management (core/resources.py)

Resources (such as radar processors, tracking channels or communication links) have **multiple instances**. The resource manager keeps the total, available, allocated and requested instances, and validates every request before passing it to the deadlock subsystem. The Resources screen provides allocator lists and slider forms.

4.8.Deadlock Detection and Avoidance (core/deadlock.py, banker.py)

Detection with a Resource Allocation Graph (RAG). Processes and resources are nodes; request edges point from a process to a resource and assignment edges from a resource to a process. A cycle indicates a deadlock for single-instance resources. The module finds cycles with a **depth-first search** and reports the cycle path.

Avoidance with the Banker's algorithm. Before granting a request, the system checks whether the resulting state is *safe*, meaning there exists an order in which all processes can finish. The implementation handles multiple instances and returns the **Safe Sequence**.

```
Need[i][j] = Max[i][j] - Allocation[i][j]

Safety(Available, Allocation, Need):
    Work = Available;  Finish[i] = False for all i;  sequence = []
    repeat:
        find i with Finish[i] == False and Need[i] <= Work
        if found:  Work += Allocation[i];  Finish[i] = True;  sequence.append(i)
        else: break
    return (all Finish[i] is True), sequence      # safe sequence or UNSAFE

Request(i, Req):
    if Req > Need[i]  -> error (exceeds declared maximum)
    if Req > Available -> process must wait
    pretend: Available -= Req; Allocation[i] += Req; Need[i] -= Req
    if Safety() is safe -> grant   else -> roll back and deny
```

4.9.Process Synchronization (core/synchronization.py)

Radar sweeps (producers) write into a **Radar Data Buffer** while tracking tasks (consumers) read from it. Access is protected by:

- **Mutex** - guarantees mutual exclusion in the critical section.
- **Counting semaphore** - tracks the number of free/occupied buffer slots (bounded buffer).
- **Race-condition demo** - with synchronization deactivated, interleaved updates corrupt the buffer in a controlled, repeatable way, making the need for synchronization visible.

4.10.Radar Simulation Layer (radar/)

simulator.py generates objects during the radar sweep (type, distance in km, speed in km/h, heading) and also accepts manually injected events. **classifier.py** maps each object to a threat level using its type, distance, speed and the user-defined threat-probability slider (0 to 1). The threat level then determines the priority and deadline of the process created for the object.

4.11. Performance Analysis (core/performance.py)

After a run, the module aggregates metrics for each algorithm so policies can be compared side by side.

Metric	Formula
Turnaround time	Completion time - Arrival time
Waiting time	Turnaround time - Burst time
Response time	First CPU time - Arrival time
CPU utilization (%)	(Busy time / Total time) x 100
Throughput	Processes completed / Total time
Context switches	Number of times the CPU is switched from one process to another
Page-fault rate	Page faults / Total page references

4.12. Persistence Layer (database/mongodb.py)

All database operations are isolated behind a dedicated `MongoDBService` class, so no other module talks to MongoDB directly. Collections store sessions, processes and performance snapshots. If the database cannot be reached, the service switches to an **in-memory fallback database**, guaranteeing crash-free offline execution. The GUI footer reports the current mode (for example *DATABASE: IN-MEMORY (MongoDB offline)*), and the Config Settings screen lets the user change the connection URI. Every run is tagged with a session identifier of the form `DRTOS-YYYYMMDD-NNNN`.

5. Graphical User Interface

The GUI is a dark-themed tactical **Command Center** built with Tkinter. A left navigation panel provides one button per subsystem, the center area displays the selected screen, and a status bar at the bottom shows the radar state and database mode. The tactical styling is centralised in `gui/theme.py`.

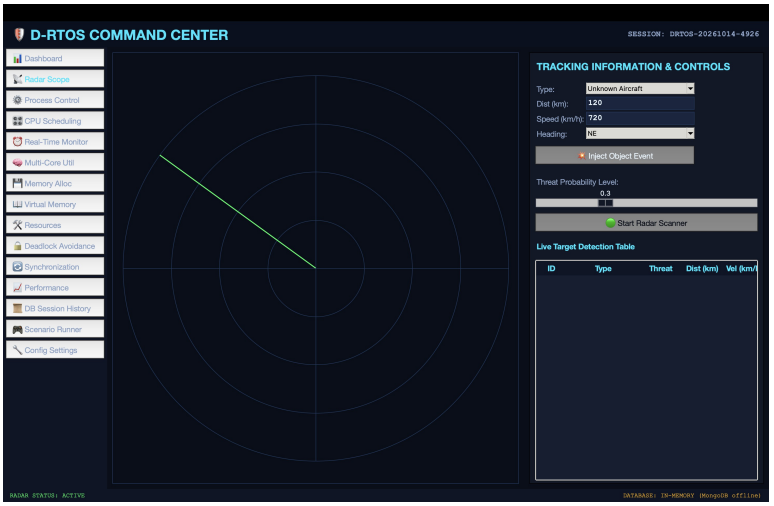


Figure 3: Radar Scope screen - PPI sweep, tracking controls and live target detection table

Radar Scope. The circular Plan Position Indicator shows the rotating sweep line and detected blips. The right panel lets the operator choose the object type, distance (km), speed (km/h) and heading, adjust the threat probability slider, press *Inject Object Event* to create a process, and *Start Radar Scanner* to begin automatic sweeps. The Live Target Detection Table lists ID, type, threat, distance and velocity of each target.

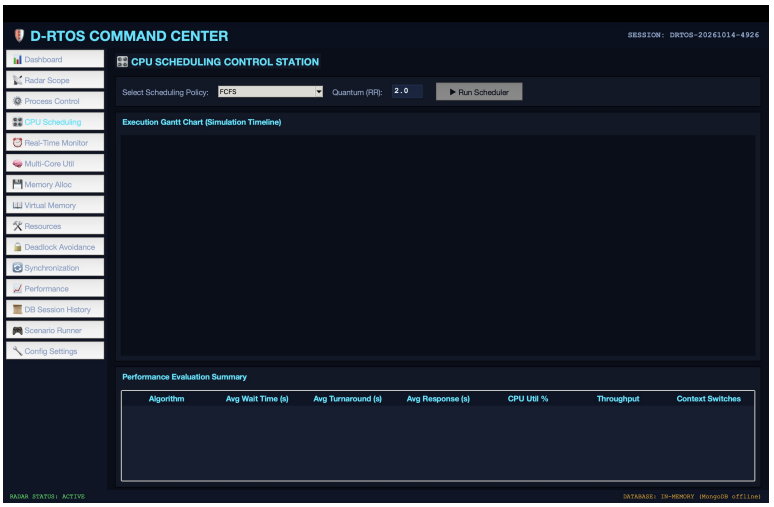


Figure 4: CPU Scheduling Control Station - policy selection, Gantt chart and performance summary

CPU Scheduling. The operator selects a policy (FCFS in the figure), sets the Round Robin quantum and presses *Run Scheduler*. The execution Gantt chart displays the timeline, and the Performance Evaluation Summary reports average waiting, turnaround and response time, CPU utilization, throughput and context switches for each algorithm run.

5.1.Screen Summary

Screen	Module	Function
Dashboard	dashboard.py	System-wide metrics overview
Radar Scope	radar_view.py	PPI sweep, blips, event injection, target table
Process Control	process_view.py	Process lists and PCB inspector
CPU Scheduling	scheduler_view.py	Gantt chart and metrics comparison
Real-Time Monitor	realtime_view.py	Deadline gauges and EDF tables
Multi-Core Util	multicore_view.py	Per-core load progress HUD
Memory Alloc	memory_view.py	Physical RAM partition graph (First/Best/Worst Fit)
Virtual Memory	paging_view.py	Address translator and FIFO/LRU stepper
Resources	resource_view.py	Allocator lists and request sliders
Deadlock Avoidance	deadlock_view.py	RAG cycle paths and Banker's safe sequence
Synchronization	synchronization_view.py	Mutex/semaphore and race-condition demo
Performance	analytics_view.py	Matplotlib bar charts comparing schedulers
DB Session History	history_view.py	Browser of stored sessions
Scenario Runner	scenario_view.py	8-phase guided walkthrough
Config Settings	settings_view.py	Database connection URI configuration

6. Installation and Usage

6.1.Requirements

- Python 3.9 or later (Tkinter is bundled with most Python distributions).
- Packages listed in `requirements.txt` (for example Matplotlib and PyMongo).
- MongoDB is **optional**; without it the application runs on the in-memory fallback.

6.2.Running the Application

```
git clone <repository-url>
cd D-RTOS
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python main.py
```

7. Testing and Verification

The `tests/` directory holds the verification suite. Because the OS engine is separate from the GUI, each algorithm can be checked with deterministic inputs against hand-computed results from the textbooks.

Area	What is verified
Process / PCB	Correct creation; only legal state transitions (NEW, READY, RUNNING, WAITING, TERMINATED)
Schedulers	Execution order and Gantt slices for each policy; metric formulas on a known process set
EDF / deadlines	Earliest deadline chosen; met/missed deadline counting
Multi-core	Tasks migrate from overloaded to underloaded cores
Memory	First/Best/Worst Fit select the expected partition; release merges holes
Paging	Address translation; FIFO and LRU fault counts on standard reference strings
Deadlock	RAG detects a known cycle; no false positive on an acyclic graph
Banker's	Textbook example returns the expected safe sequence; unsafe requests are denied
Synchronization	Protected buffer stays consistent; unprotected run shows corruption
Database	Fallback to in-memory store when MongoDB is unreachable; save/load round trip

The test suite can be executed from the project root with `python -m unittest discover tests` (or `pytest` if the tests are written for it).

8. Mapping to Operating-System Syllabus Outcomes

The table shows how each syllabus topic is covered by the project, which can be used to justify the 40-mark weightage.

Syllabus topic	Concept demonstrated	Implemented in
Process management	PCB, process states, queues, context switching	process.py, pcb.py, process_manager.py
CPU scheduling	FCFS, SJF, Priority, RR, Preemptive Priority, SRTF, MLQ	scheduler.py
Real-time systems	EDF, deadlines, hard real-time monitoring	realtime.py
Multiprocessor scheduling	Per-core queues, load balancing, migration	multicore.py
Memory management	Contiguous allocation, fragmentation, fit strategies	memory.py
Virtual memory	Paging, address translation, FIFO/LRU replacement	paging.py
Deadlocks	RAG detection, Banker's avoidance, safe sequence	deadlock.py, banker.py
Synchronization	Mutex, counting semaphore, bounded buffer, race condition	synchronization.py
Performance evaluation	Waiting, turnaround, response, utilization, throughput	performance.py
Persistence	Database service layer with fault-tolerant fallback	mongodb.py

9. Results and Observations

Running the simulator on the same set of radar-generated processes allows a direct comparison of policies. The following general behaviour, consistent with OS theory, can be observed in the Performance Evaluation Summary and the analytics charts:

- SJF and SRTF typically give the lowest average waiting time, while FCFS shows longer waiting when a long job arrives first.
- Round Robin gives the best response time but increases context switches as the quantum is reduced.
- Priority-based and MLQ policies serve critical threats first, at the cost of delaying low-priority routine tracking.
- EDF meets deadlines whenever the task set is schedulable; the monitor flags a miss when the system is overloaded.
- LRU usually gives fewer page faults than FIFO on locality-rich reference strings, and FIFO may show Belady's anomaly.
- With synchronization disabled the radar buffer becomes inconsistent; with mutex and semaphore enabled it remains correct.

10. Limitations and Future Scope

10.1.Limitations

- Time is simulated; there are no real hardware interrupts or kernel-level context switches.
- Radar objects and threats are fictional and generated by a simplified model.
- Inter-process communication (pipes, message queues, shared memory) is not included.
- The Tkinter GUI is a desktop application only.

10.2.Future Scope

- Additional real-time policies such as Rate Monotonic Scheduling and priority inheritance for priority inversion.
- Multilevel Feedback Queue scheduling and Linux CFS-style fair scheduling.
- File-system and disk-scheduling modules, IPC mechanisms and a web-based dashboard.

11. Conclusion

D-RTOS demonstrates that the main topics of an Operating Systems course can be integrated into a single, coherent simulation. By turning radar events into processes and driving them through scheduling, multi-core execution, memory management, resource control, deadlock avoidance and synchronization, the project shows how these mechanisms cooperate in a real-time system. The modular layered design, the fault-tolerant database layer and the interactive command-center GUI make the system easy to test, extend and present, and satisfy the learning objectives of the 40-mark project.

References

1. A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Wiley.
2. W. Stallings, *Operating Systems: Internals and Design Principles*, Pearson.
3. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, Pearson.
4. C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment", *Journal of the ACM*, 1973.
5. E. W. Dijkstra, "Cooperating Sequential Processes", 1965 (semaphores); "Solution of a Problem in Concurrent Programming Control", 1965.
6. Python Software Foundation, Python documentation (tkinter); Matplotlib documentation; MongoDB documentation.